

Visione Artificiale (Computer Vision)

Liliana Lo Presti

Reference

- API documentation is at the link <https://docs.opencv.org/4.7.0/> (you may need to change the library version)
- Documentation refers to the main modules of the library:
 - core: Core functionality
 - imgproc: Image Processing
 - imgcodecs: Image file reading and writing
 - videoio: Video I/O
 - highgui: High-level GUI
 - video: Video Analysis
 - calib3d: Camera Calibration and 3D Reconstruction
 - features2d: 2D Features Framework
 - objdetect: Object Detection
 - dnn: Deep Neural Network module
 - ml: Machine Learning
 - flann: Clustering and Search in Multi-Dimensional Spaces
 - photo: Computational Photography
 - stitching: Images stitching
 - gapi: Graph API

OpenCV – Installation and notes

- Documentation is mainly in C++ but wrapper functions for Python are indicated as well.
- The library can be installed in your Python environment by using "*pip install opencv-python*"
- The command will install an unofficial pre-built OpenCV package for Python
- Once the library is installed, you can **import** it
- *openCV 2 is written in C++* and uses several libraries, especially to handle image formats and video codec.
- Some functionalities may not work if the pre-built openCV library has been compiled with external libraries that are actually missing in your machine.
- When this occurs, you need to check the openCV's Build information and check if you are missing some of them!! In general, issues are solved by installing the missing libraries.
- You need to check specifically the libraries for: GUI (since windows will not work if there are issues), Video I/O (since you will have troubles opening videos).

OpenCV – Check build information

- `import cv2`
- `print(cv2.getBuildInformation())` #this command will print the building information

Images

- Images are treated as N-channels numpy array of 8 bit unsigned int.
- Color images are made of 3 channels and openCV stores them as BGR (not RGB). This is important to remember when you wanna process them or use multiple libraries (es. imageio).
- To deal with the images, we will also import numpy
- Images can be stored in different formats, depending also on the compression level used to store the image.
- Common formats are (many other exist!):
 - Windows bitmaps – *.bmp, *.dib
 - JPEG files – *.jpeg, *.jpg
 - Portable Network Graphics – *.png
 - TIFF files – *.tiff, *.tif

Images

The loaded image will be a C-dim numpy array of size HxWxC where H = height, W=width, C=Channels
You need to think it as a 2D pixel arrays for C color channels.

In the following: **red = row index**, **blue = column index**, **orange is the channel** (0=blue, 1=green, 2=red)

```
im = cv2.imread("Lenna.png")
```

```
print("data type: ", type(im))
```

```
print("image size: ", im.shape)
```

```
print("element data type: ", im.dtype)
```

```
print("First pixel, blue channel: ", im[0, 0][0])
```

```
print("Last pixel in first row, red channel: ", im[0, im.shape[1] - 1, 2])
```

```
print("First pixel in last row, green channel: ", im[im.shape[0] - 1, 0, 1])
```

```
print("Last pixel in last row, red channel: ", im[im.shape[0] - 1, im.shape[1] - 1, 2])
```

Images

We can also visualize an image by using the openCV method `imshow()`.

GUI in opencv has methods for creating windows, destroying windows. Windows are characterized by a name, which is a string used as ID for the window.

```
cv2.namedWindow(window_name, flag)
cv2.destroyWindow(window_name)
cv2.destroyAllWindows()
```

Documentation for the meaning of flag:

https://docs.opencv.org/4.7.0/d7/dfc/group__highgui.html#ga5afdf8410934fd099df85c75b2e0888b

The method `imshow` will create a window and display the image. However, we need to give time to the system for the rendering.

Hence, we need to add a pause. This can be accomplished by using the method `cv2.waitKey(delay)`.

`delay = 0` means that it will pause until a key is not pressed (focus on the window first). `delay` is expressed in ms

Note: be aware that within the notebook, `destroyAllWindows` may not work. Try it in a separate project.

Images

```
sub_im = im[250:, 200:]  
cv2.imshow("Lenna", im)  
cv2.imshow("Lenna_sub", sub_im)  
cv2.waitKey(0)  
#cv2.destroyAllWindows()
```

When we define RoI (sub-images) **we are not creating a copy** but a reference to a portion of the image. To make a deep copy use:

```
im_deep_copy = im.copy()
```


Writing Images and Data format

```
ret = cv2.imwrite("new_lenna.png", im_deep_copy)
print(ret)
```

- To be visualized or stored, image values must be in the range [0 255] and the data type must be uint8.

- If instead we wanna manipulate images, we need to be careful to the data type! In fact, it is better to cast the data type to float or double before manipulating the image and the casy back to the uint8 format.

Sometimes, we need to renormalize the data in the range [0 255].

Manipulating Images

```
black_im = np.zeros((512, 256), np.uint8)
white_im = 255*np.ones((512, 256), np.uint8)
gray_im = 0.5*(black_im.astype(np.float32) + white_im.astype(np.float32) )
gray_im = gray_im.astype(np.uint8) #without the cast, data are interpreted in the wrong way...
```

```
new_image = np.random.randint(0, 1000, (256, 256, 3))
new_image = cv2.normalize(new_image, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
```

Videos

- Handling videos is generally very complex but openCV provides tools for making it easier.
- **A video is a sequence of images**, called frames.
- The **number of frames per second** is defined frame rate of **fps** (frames per second). Typical value are 25 (in EU) or 30 (in US) but other values are common nowadays.
- All frames must have **all the same size**. Typical video resolution (WxH) are:
 - 1280 x 720 (HD or 720p)
 - 1920 x 1080 (Full HD or 1080p)
 - 3840 x 2160 (4K or Ultra HD or 2160p)
- Storing a 60 seconds color video in HD at 25 fps requires saving 1500 frames on HDD.
- Each frame occupies 1280x720x3 byte (about 2.8 MB). Storing this short video would require 4.2 GB.
- However, many of these frames will look pretty similar. Thus, it is possible to use **compression techniques** to save space. Compression is also **important for streaming** since it reduces the bandwidth required to stream the video.
- Moreover, to enhance compatibility among devices and platforms, **compressed videos must be coded using suitable standard**.

Digital Videos

- A video is a sequence of images, all of the same size (HxW), called frames
- A video stream is characterized by:
 - Frame rate, i.e. number of frames per seconds. Two standards are used for customer devices:
 - **PAL** used in EU, Asia, Australia specifying 25 fps
 - **NTSC** used in US, Canada, Japan specifying almost 30 fps
 - Aspect ratio: the ratio **width/height**, Typical formats are **4:3** and **16:9**
 - Video resolution, in the sense of typical frame size:
 - 1280 x 720 (HD or 720p)
 - 1920 x 1080 (Full HD or 1080p)
 - 3840 x 2160 (4K or Ultra HD or 2160p)

Videos

- **Video codec** refers to **compression standards**. The video coding process requires a coding step (done through the **encoder**) to compress data and a decoding step (through a **decoder**) to approximate the original video (when compression determines a loss of information).
- **Codec** and **video format** are not the same:
 - **Codec** is the compression-decompression algorithm. The **most used codecs** include:
 - H.264 (especially for streaming, is also called AVC),
 - H.265 (called also HEVC),
 - VP9,
 - AV1.
 - **Video format** indicates the type of file, which is like a container for the compressed video but also for other data, such as audio or subtitles. Common video formats are:
 - MKV,
 - MP4 (also known as MPEG 4).
- Due to the compression algorithm, it is difficult to read a random frame from a video. Indeed, the compression of this frame may involve also information extracted from previous frames (depending on the compression algorithm).

Videos

- To handle videos, openCV uses external libraries. If you miss some codec, you may not be able to open some video formats
- The steps to use a video are similar to those required when handling files:
 - open the video;
 - read frame-by-frame (for instance, within a for loop) till the end of the video;
 - close the video (and free resources allocated).
- In openCV, videos are handled by an object of the class `VideoCapture`.
- The video can be opened by the constructor of the object or through the method `open()`.
- Frames are read by the method `read()`.
- The method `release()` allows closing the file.

Videos

```
video = cv2.VideoCapture("Tracking.mp4")
if video is None or not video.isOpened():
    raise IOError("video not found")
else:
    ret, img = video.read()
    while ret:
        cv2.imshow("frame", img)
        cv2.waitKey(1)
        ret, img = video.read()

print("number of frames ", video.get(cv2.CAP_PROP_FRAME_COUNT))
video.release()
```

Streaming from Cameras

```
ID = 0 # webcam
video = cv2.VideoCapture(ID)
if video is None or not video.isOpened():
    raise IOError("video not found")
else:
    print("frame width ", video.get(cv2.CAP_PROP_FRAME_WIDTH ))
    print("frame height ", video.get(cv2.CAP_PROP_FRAME_HEIGHT ))
    print("frame rate ", video.get(cv2.CAP_PROP_FPS ))
    ret, img = video.read()
    key = ''
    while ret and key!=ord('q'):
        cv2.imshow("frame", img)
        key = cv2.waitKey(1)
        ret, img = video.read()
    video.release()
```


Color Space

- https://docs.opencv.org/3.4/d8/d01/group__imgproc__color__conversions.html
- To change the color space, we use `cv2.cvtColor()`

`imRGB = cv2.cvtColor(imBGR, cv2.COLOR_BGR2RGB)`

`imgray = cv2.cvtColor(imBGR, cv2.COLOR_BGR2GRAY)`